Отчёт по лабораторной работе №2

Основные модели

Вакутайпа Милдред

Содержание

1 Цель работы **5**

2 Задание **6**

3 Теоретическое введение **7**

4 Выполнение лабораторной работы **9**

4.1 Модель SIR 9

4.2 Модель Лотки-Вольтерры 24

5 Выводы **47**

Список литературы **48**

Список иллюстраций

4.1 Репродуктивное число с исходными параметрами 17
4.2 Количество инфицированных 17
4.3 Экпоненциальный рост 18
4.4 Динамика эпидемии 18
4.5 Динамика эпидемии в процентах 19
4.6 Фазовая траектория 19
4.7 Итоговый график 20
4.8 статистика набора параметров 22
4.9 Работа с SIR в jupyter 23
4.10 сравнение количества зараженных 24
4.11 сравнение репродуктивного числа 24
4.12 Производные популяции 36
4.13 Динамика популяции 36
4.14 Фазовый портрет lv 37
4.15 Относительные темпы роста 37
4.16 Спектральный анализ 38
4.17 Итоговая график lv 38
4.18 Статистика модели lv с набором параметров 43
4.19 Работа в jupyter 44
4.20 Сравнение фазовых портретов 45
4.21 Сравнение попляции жертв 45
4.22 Сравнение популяции жищков 46

Список таблиц

1 Цель работы

Цель данная работа – освоить работу с профессиональные инструменты, научиться работать с фундаментальными моделями SIR и Лотки-Волтерра.

2 Задание

1. Создать рабочий каталог для кода.
2. Установить необходимые пакеты.
3. Выполнить предложенный код.
4. Преобразовать код в литературный стиль.
5. Сгенерировать из литературного кода: — чистый код; — jupyter notebook; — документацию в формате Quarto.
6. Выполнить код из jupyter notebook.
7. Интегрировать документацию в формате Quarto в отчёт.
8. Добавить в код в литературном стиле вычисление для набора параметров.
9. Сгенерировать из литературного кода с параметрами: — чистый код; — jupyter notebook; — документацию в формате Quarto.
10. Выполнить код из jupyter notebook с параметрами.
11. Интегрировать документацию с параметрами в формате Quarto в отчёт.

3 Теоретическое введение

Модель SIR есть классическая и фундаментальная математическая модель эпидемиологии, описывающая распространение инфекционного заболевания в закрытой популяции.

Модель SIR делит всю популяцию на три взаимосвязанные группы (компартменты), что отражено в её названии: — ⊠ — Susceptible (Восприимчивые): люди, которые не болели, не имеют иммунитета и могут заразиться. — ⊠ — Infectious (Инфицированные/Заразные): люди, которые в данный момент больны и могут передавать инфекцию. — ⊠ — Recovered (Выздоровевшие/Удаленные): люди, которые переболели и приобрели иммунитет (или умерли). Они больше не участвуют в процессе передачи.

Ключевая идея модели Лотки-Вольтерры заключается в, том что она демонстрирует, как даже простая система взаимодействий может порождать сложные колебательные режимы, объясняя циклические изменения численности в природных экосистемах.

Модель строится на следующих упрощающих предположениях: — Закрытая система: популяции изолированы, нет миграции. — Неограниченные ресурсы для жертв: в отсутствие хищников жертвы растут экспоненциально. — Линейная функциональная реакция: вероятность встречи хищника и жертвы пропорциональна произведению их численностей. — Постоянные параметры: коэффициенты взаимодействия не меняются во вре- мени. — Отсутствие внутривидовой конкуренции: нет конкуренции за ресурсы внутри вида. — Хищники питаются только жертвами: нет альтернативных источников пищи. —

Отсутствие временных задержек: все процессы происходят мгновенно.

4 Выполнение лабораторной работы

До того, как начала выполнить работу я создала кактлог «project» в котором храняется все скрипты, графики, изображения итп. В этом же каталоге я импортировала все необходимые пакеты следующим скриптом:

```
using Pkg
Pkg.activate(".")
Pkg.add("DifferentialEquations")
Pkg.add("SimpleDiffEq")
Pkg.add("Tables")
Pkg.add("DataFrames")
Pkg.add("StatsPlots")
Pkg.add("LaTeXStrings")
Pkg.add("Plots")
Pkg.add("BenchmarkTools")
Pkg.add("Statistics")
Pkg.add("FFTW")
```

4.1 Модель SIR

Далее я выполнила предложенный код для модели SIR:

```julia
using DrWatson
@quickactivate "project"

using DifferentialEquations
using SimpleDiffEq
using Tables
using DataFrames
using StatsPlots
using LaTeXStrings
using Plots
using BenchmarkTools

script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))

function sir_ode!(du, u, p, t)
    (S, I, R) = u
    (⊠, c, ⊠) = p
    N = S + I + R
    @inbounds begin
        du[1] = -⊠ * c * I / N * S
        du[2] = ⊠ * c * I / N * S - ⊠ * I
        du[3] = ⊠ * I
    end
    nothing
end
```

```
□t = 0.1
tmax = 40.0
tspan = (0.0, tmax)
u0 = [990.0, 10.0, 0.0]  # S, I, R
p = [0.05, 10.0, 0.25] # □, c, □

# □□□□□□ □□□□□□□□ □□□□□□□□□□□□□□□ □□□□□
R0 = (p[2] * p[1]) / p[3]

prob_ode = ODEProblem(sir_ode!, u0, tspan, p)
sol_ode = solve(prob_ode, dt = □t)

df_ode = DataFrame(Tables.table(sol_ode'))
rename!(df_ode, ["S", "I", "R"])
df_ode[!, :t] = sol_ode.t
df_ode[!, :N] = df_ode.S + df_ode.I + df_ode.R

println("□□□□□□□□□ □□□□□□ SIR:")
println("□ (□□□□□□□□□□□ □□□□□□□□□) = ", p[1])
println("c (□□□□□□□ □□□□□ □□□□□□□□□) = ", p[2])
println("□ (□□□□□□□□ □□□□□□□□□□□□□) = ", p[3])
println("R0 = c * □ / □ = ", round(R0, digits=3))
println("□□□□□□□ □□□□□□□□□□□□□□□□ □□□□□□□ = ", round(1/p[3], digits=2), "
println("□□□□□□□□□ □□□□□□□: S0 = ", u0[1], ", I0 = ", u0[2], ", R0 =",

plt1 = @df df_ode plot(:t,
    [:S :I :R],
    label=[L"S(t)" L"I(t)" L"R(t)"],
```

```
    xlabel="□□□□□, □□□",
    ylabel="□□□□□□□□□□ □□□□□",
    title="□□□□□□ SIR: □□□□□□□□ □□□□□□□□",
    linewidth=2,
    legend=:right,
    grid=true,
    size=(800, 500))

annotate!(plt1, maximum(df_ode.t) * 0.7, maximum(df_ode.N) * 0.8,
    text("□□□□□□□□□:\n□ = $(p[1])\nc = $(p[2])\n□ = $(p[3])\nR0 =$(rou
    8, :left))

plt2 = @df df_ode plot(:t, :I,
    label=L"I(t)",
    xlabel="□□□□□, □□□",
    ylabel="□□□□□□□□□□ □□□□□□□□□□□□□",
    title="□□□□□□□□ □□□□□ □□□□□□□□□□",
    color=:red,
    linewidth=2,
    fill=(0, 0.3, :red),
    grid=true,
    size=(800, 400))

peak_idx = argmax(df_ode.I)
peak_time = df_ode.t[peak_idx]
peak_value = df_ode.I[peak_idx]
vline!(plt2, [peak_time], color=:black, linestyle=:dash, label=false
annotate!(plt2, peak_time, peak_value * 1.05,
```

```
    text("□□□: $(round(peak_value, digits=1)) □□ $(round(peak_time,di
    8, :top))

plt3 = @df df_ode plot(:t, :I,
    label=L"I(t)",
    xlabel="□□□□□, □□□",
    ylabel="□□□□□□□□□□ □□□□□□□□□□□□□□ (□□□. □□□□□□□)",
    title="□□□□□□□□□□□□□□□□ □□□□ (□□□. □□□□□)",
    yscale=:log10,
    color=:red,
    linewidth=2,
    grid=true,
    size=(800, 400))

plt4 = @df df_ode plot(:t,
    [:S :I :R] ./ df_ode.N .* 100,
    label=[L"S(t)/N" L"I(t)/N" L"R(t)/N"],
    xlabel="□□□□□, □□□",
    ylabel="□□□□ □□□□□□□□□, %",
    title="□□□□□□□□ □□□□□□□□ (□ □□□□□□□□□)",
    linewidth=2,
    legend=:right,
    grid=true,
    size=(800, 500))

if R0 > 1
    herd_immunity_threshold = (1 - 1/R0) * 100
    hline!(plt4, [herd_immunity_threshold], color=:purple,linestyle=
```

```
    label="□□□□□ □□□□□□□□□□□□□ □□□□□□□□□□□($(round(herd_immunity_threshold
    linewidth=1.5)
end

plt5 = plot(df_ode.S, df_ode.I,
    label="□□□□□□□ □□□□□□□□□□□",
    xlabel=L"S(t)",
    ylabel=L"I(t)",
    title="□□□□□□□ □□□□□□□ SIR □□□□□□",
    color=:blue,
    linewidth=2,
    grid=true,
    size=(800, 500),
    legend=:topright)

for i in 1:50:length(df_ode.S)-1
    plot!(plt5, [df_ode.S[i], df_ode.S[i+1]], [df_ode.I[i], df_ode.
    arrow=:closed, color=:blue, alpha=0.5, label=false)
end

df_ode[!, :Re] = R0 .* df_ode.S ./ df_ode.N

plt6 = @df df_ode plot(:t, :Re,
    label=L"R_e(t)",
    xlabel="□□□□□, □□□",
    ylabel=L"R_e",
    title="□□□□□□□□ □□□□□□□□□□□□□ □□□□□□□□□□□□□□□□ □□□□□",
    color=:green,
```

```
    linewidth=2,
    grid=true,
    size=(800, 400))

# □□□□□□□□□□□□□□□ □□□□□□ □□ □□□□□□□ 1
hline!(plt6, [1.0], color=:red, linestyle=:dash, label="□□□□□ □□□□□□□□□

cross_idx = findfirst(x -> x < 1, df_ode.Re)
if !isnothing(cross_idx) && cross_idx > 1
    cross_time = df_ode.t[cross_idx]
    vline!(plt6, [cross_time], color=:black, linestyle=:dash, label=
    annotate!(plt6, cross_time, 1.2,
        text("Rₑ<1 □ $(round(cross_time, digits=1)) □□□", 8, :left))
end

plt7 = plot(layout=(2, 3), size=(1200, 800))

plot!(plt7[1], df_ode.t, df_ode.S, label=L"S(t)", color=1, linewidt
plot!(plt7[2], df_ode.t, df_ode.I, label=L"I(t)", color=2, linewidt
plot!(plt7[3], df_ode.t, df_ode.R, label=L"R(t)", color=3, linewidt

plot!(plt7[4], df_ode.t, df_ode.I, label=L"I(t)", color=2, linewidt
plot!(plt7[5], df_ode.S, df_ode.I, label=false, color=4, linewidth=
plot!(plt7[6], df_ode.t, df_ode.Re, label=L"R_e", color=:green, lin

savefig(plt1, plotsdir(script_name, "sir_main.png"))
savefig(plt2, plotsdir(script_name, "sir_infected.png"))
savefig(plt3, plotsdir(script_name, "sir_log_scale.png"))
```

```
savefig(plt4, plotsdir(script_name, "sir_percentages.png"))
savefig(plt5, plotsdir(script_name, "sir_phase_portrait.png"))
savefig(plt6, plotsdir(script_name, "sir_effective_R.png"))
savefig(plt7, plotsdir(script_name, "sir_panel.png"))

println("\n□□□□□□□□ □□□□□□□:")
@benchmark solve(prob_ode, dt = □t)
# □□□□□□□□□□□□□□ □□□□□□
println("\n=== □□□□□□ □□□□□□□□□□□ ===")
println("□□□□□ □□□□□□□□□□□ □□□□□□□□□ (□□□□□□□□): N = ", round(df_ode.N[1],
println("□□□□□□□ □□□□□ □□□□□□□□□□: I_max = ", round(peak_value, digits=1
println("□□□□□ □□□□□□□□□□ □□□□: t_peak = ", round(peak_time, digits=1),
println("□□□□□□□□ □□□□□ □□□□□□□□□□□□: R(∞) = ", round(df_ode.R[end], digi
println("□□□□ □□□□□□□□□□□□: ", round(df_ode.R[end]/df_ode.N[1]*100, dig

if R0 > 1
    println("\n□□□□□□□□□□□□□ □□□□□□:")
    println(" - □□□□□ □□□□□□□□□□□□□ □□□□□□□□□□: ", round((1-1/R0)*100, dig
    println(" - □□□□□□□□□□□□□□ □□□ □□□ S/N = 1/R0 = ", round(1/R0, digits
end
```

Используя скрит для преобразовния кода в литературном стиле с прошлой работы я сгенерировала чистый код, документацию в формате Quarto, jupyter notebook. Потом я выполнила код в jupyter, чтобы проверить работу. В результате получила следующие графики:

Репродуктивное число убывает и значит эпидемия затухает:

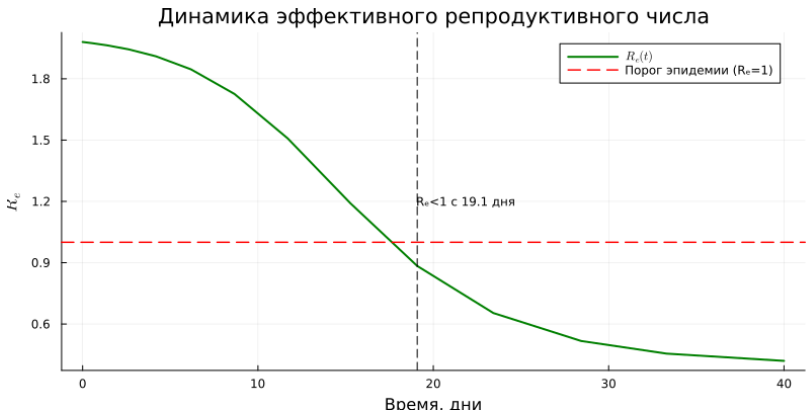

Рисунок 4.1: Репродуктивное число с исходными параметрами

По графике видно, что максимальное количество инфицированных было на 19 день эпидемии и составила 155

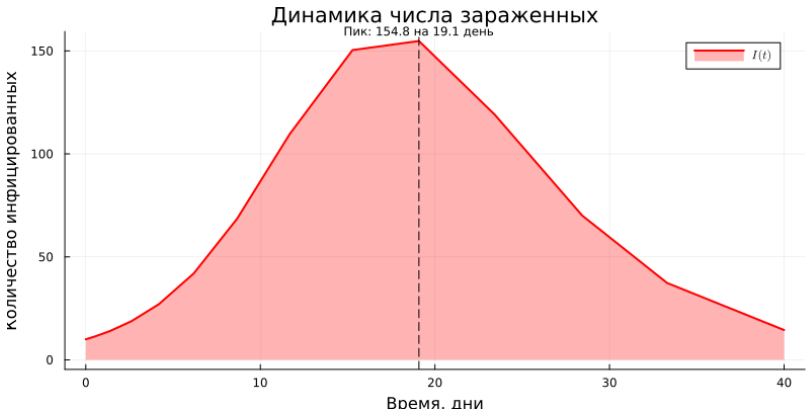

Рисунок 4.2: Количество инфицированных

Экспоненциальный график показывает рост количество инфицированных

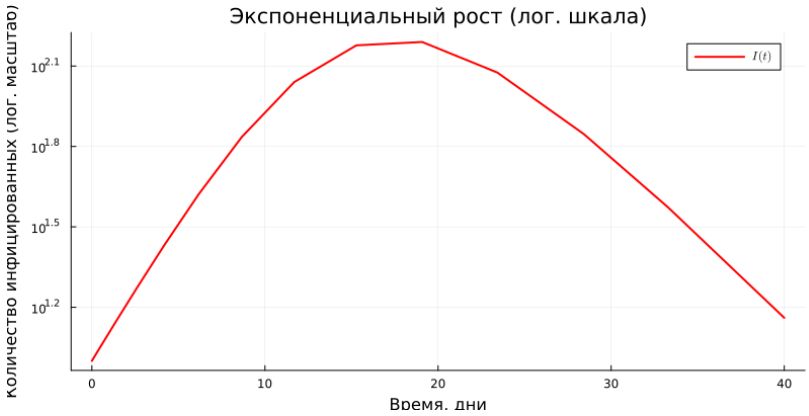

Рисунок 4.3: Экпоненциальный рост

В соответствие с репродуктивным числом число загражненных убывает и число выздоровших вырастает

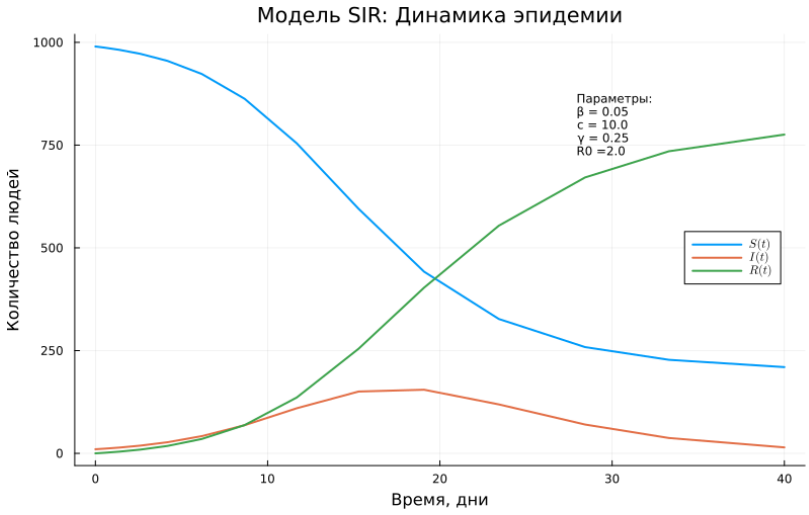

Рисунок 4.4: Динамика эпидемии

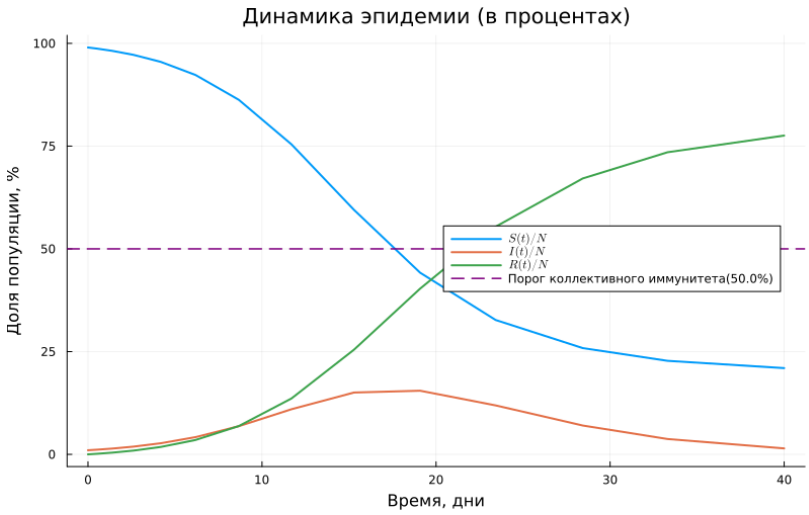

Рисунок 4.5: Динамика эпидемии в процентах

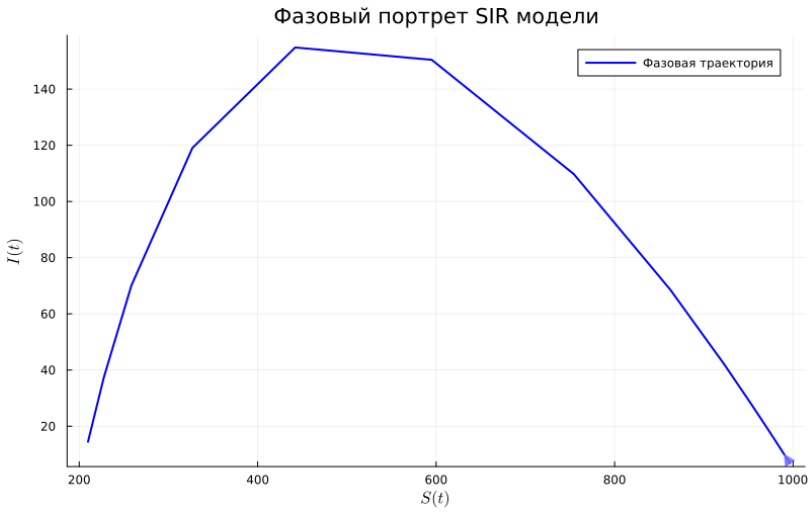

Рисунок 4.6: Фазовая траектория

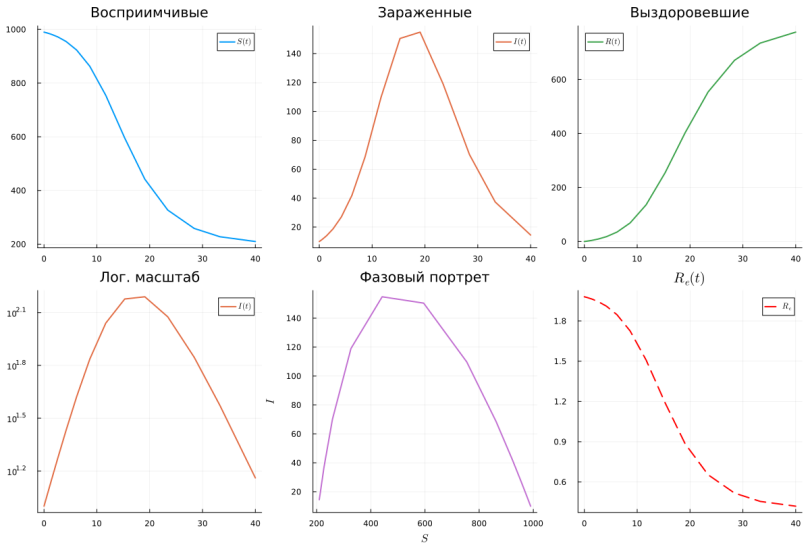

Рисунок 4.7: Итоговый график

Я добавила в код в литературном стиле набор параметров:

```
parameter_sets = [
    [0.05, 20.0, 0.25],   # [redacted]
    [0.10, 20.0, 0.25],   # [redacted] ([redacted])
    [0.15, 20.0, 0.25],   # [redacted]
    [0.10, 15.0, 0.25],   # [redacted]
    [0.10, 25.0, 0.25],   # [redacted]
    [0.10, 20.0, 0.15],   # [redacted]
    [0.10, 20.0, 0.35],   # [redacted]
]

# [redacted]
results = []

# [redacted]
for (idx, p) in enumerate(parameter_sets)
```

```
# □□□□□□ □□□□□□□□ □□□□□□□□□□□□□□□ □□□□□
R0 = (p[2] * p[1]) / p[3]

# □□□□□□□ □ □□□□□□ ODE □□□□□□
prob_ode = ODEProblem(sir_ode!, u0, tspan, p)
sol_ode = solve(prob_ode, dt = □t)

# □□□□□□□□□□□ □ DataFrame
df_ode = DataFrame(Tables.table(sol_ode'))
rename!(df_ode, ["S", "I", "R"])
df_ode[!, :t] = sol_ode.t
df_ode[!, :N] = df_ode.S + df_ode.I + df_ode.R
df_ode[!, :R0] .= R0
df_ode[!, :Re] = R0 .* df_ode.S ./ df_ode.N
df_ode[!, :□□□□□□□□] .= "□□□□□ $idx"

# □□□□□□□□□ □□□□□□□□□□
push!(results, df_ode)

# □□□□□□□ □□□□□□□□□ □□□ □□□□□ □□□□□□□
println("\n=== □□□□□ □□□□□□□□□□ $idx ===")
println("□ (□□□□□□□□□□□ □□□□□□□□□) = ", p[1])
println("c (□□□□□□□ □□□□□ □□□□□□□□□) = ", p[2])
println("□ (□□□□□□□□ □□□□□□□□□□□□□) = ", p[3])
println("R0 = c * □ / □ = ", round(R0, digits=3))
println("□□□□□□□ □□□□□□□□□□□□□□□□□ □□□□□□□ = ", round(1/p[3], digits=2

# □□□□□□ □□□ □□□□□ □□□□□□□
```

```
    peak_idx = argmax(df_ode.I)
    peak_time = df_ode.t[peak_idx]
    peak_value = df_ode.I[peak_idx]

    println("□□□□□□□ □□□□□ □□□□□□□□□□: I_max = ", round(peak_value, digi
    println("□□□□□ □□□□□□□□□□ □□□□: t_peak = ", round(peak_time, digits=
    println("□□□□□□□□ □□□□□ □□□□□□□□□□□□: R(inf) = ", round(df_ode.R[end]
    println("□□□□ □□□□□□□□□□□□: ", round(df_ode.R[end]/df_ode.N[1]*100,

    if R0 > 1
        println("□□□□□□□□□□□□□ □□□□□ □□□□□□□□□□□□□ □□□□□□□□□□: ", round((1-1
    end
end
```

выполнила скрипт и получила такую статистику

```
=== СВОДНАЯ СТАТИСТИКА ===
7×9 DataFrame
 Row │ сценарий  β        c        γ        R0       пик_зараженных  время_пика  итог_пе
реболевших  доля_переболевших
     │ String    Float64  Float64  Float64  Float64  Float64         Float64     Float64
          Float64
─────┼──────────────────────────────────────────────────────────────────────────────────
──────────────────────────────
   1 │ Набор 1      0.05     20.0     0.25   4.0            400.627     8.27961
   979.932           97.9932
   2 │ Набор 2      0.1      20.0     0.25   8.0            613.361     4.02517
   999.573           99.9573
   3 │ Набор 3      0.15     20.0     0.25  12.0            707.534     2.48857
   999.924           99.9924
   4 │ Набор 4      0.1      15.0     0.25   6.0            533.797     5.32615
   997.367           99.7367
   5 │ Набор 5      0.1      25.0     0.25  10.0            657.737     2.81711
   999.877           99.9877
   6 │ Набор 6      0.1      20.0     0.15  13.3333         730.576     3.83199
   996.348           99.6348
   7 │ Набор 7      0.1      20.0     0.35   5.71429        513.363     3.54229
   996.669           99.6669
```

Рисунок 4.8: статистика набора параметров

Проверила работу в jupyter и создала график статистики

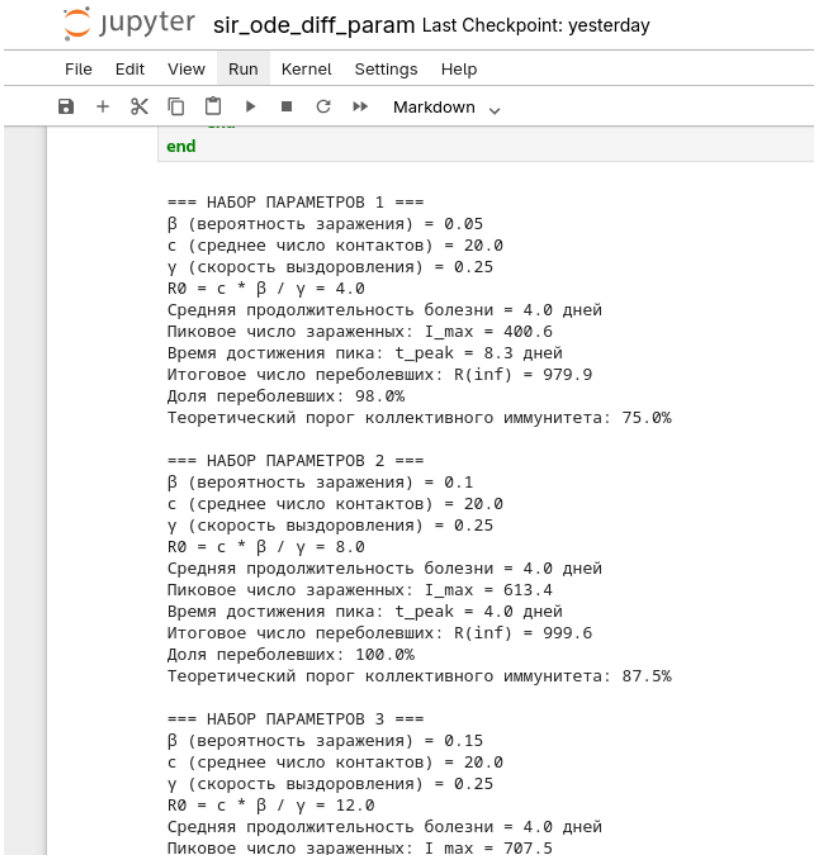

Рисунок 4.9: Работа с SIR в jupyter

Потом я сохранила графики которые сравнивают репродуктивное число и количество зараженных.

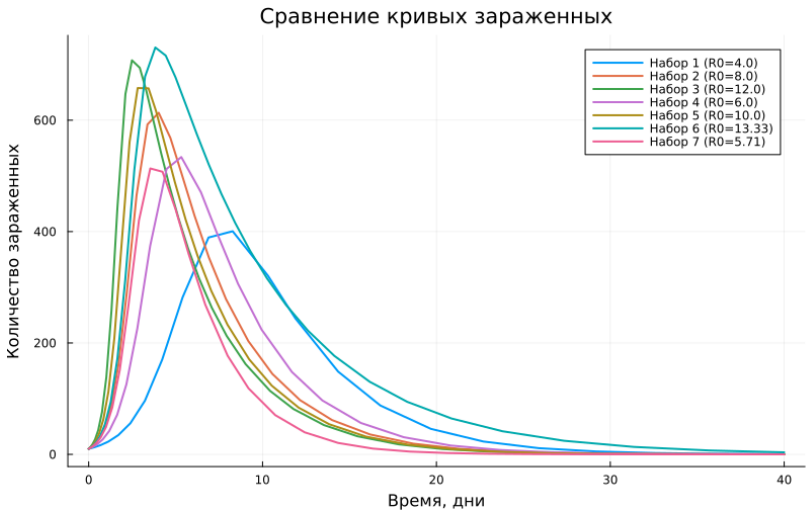

Рисунок 4.10: сравнение количества зараженных

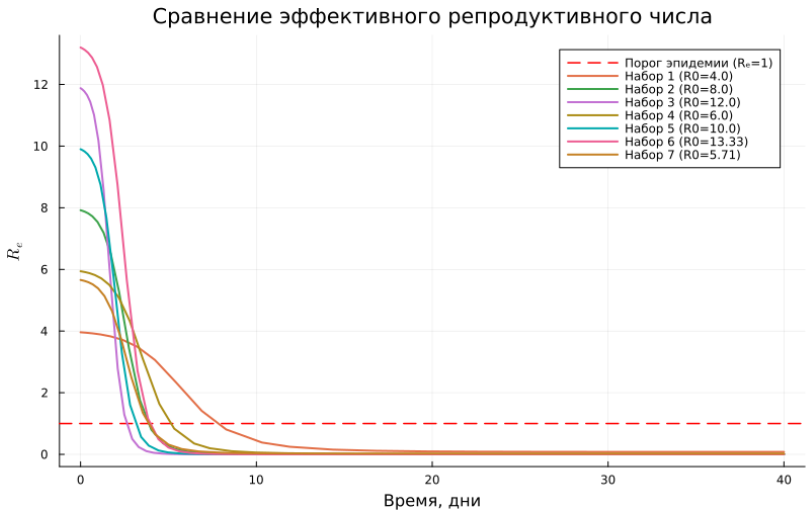

Рисунок 4.11: сравнение репродуктивного числа

Такие модели с наборами праметров важны для сравнении статистики эпидемии разных регионов и понимание эпидемии.

4.2 Модель Лотки-Вольтерры

Как и в модели SIR, я выполнила предложенный код

```julia
using DrWatson
@quickactivate "project"

using DifferentialEquations
using DataFrames
using StatsPlots
using LaTeXStrings
using Plots
using Statistics
using FFTW


script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))

# □□□□□□□□ □□□□□□ □□□□□-□□□□□□□□□
"""
□□□□□□ □□□□□-□□□□□□□□□ (□□□□□□-□□□□□□)
□□□□□□□ □□□□□□□□□:
dx/dt = □x - □xy
dy/dt = □xy - □y
# □□□□□□□□□ □□□□□□□□□ □□□□□
# □□□□□□□□□ □□□□□□□□□ □□□□□□□□
□□□:
x - □□□□□□□□□ □□□□□ (□□□□□□□□, □□□□□)
y - □□□□□□□□□ □□□□□□□□ (□□□□□□□□, □□□□)
□ - □□□□□□□□□□□□□ □□□□□□□ □□□□□ (□ □□□□□□□□□□ □□□□□□□□)
```

```
□ - □□□□□□□□□□□ □□□□□□□□□ □□□□□ □□□□□□□□□□
□ - □□□□□□□□□□□ □□□□□□□□□ □□□□□□□□□ □□ □□□□□ □□□□□□□□□ □□□□□□
□ - □□□□□□□□□□□□ □□□□□□□□□□ □□□□□□□□□ (□ □□□□□□□□□□ □□□□□□)
"""

function lotka_volterra!(du, u, p, t)
    x, y = u # x - □□□□□□, y - □□□□□□□
    □, □, □, □ = p # □□□□□□□□□ □□□□□□
    @inbounds begin
    du[1] = □*x - □*x*y # □□□□□□□□□ □□□ □□□□□
    du[2] = □*x*y - □*y  # □□□□□□□□□ □□□ □□□□□□□□
    end
    nothing
end

# □□□□□□□□□ □□□□□□ □ □□□□□□□□□ □□□□□□□
# □□□□□□□□□□□□ □□□□□□□□□ □□ □□□□□□□□□□

p_lv = [0.1, # □: □□□□□□□□ □□□□□□□□□□□ □□□□□
    0.02, # □: □□□□□□□□ □□□□□□□□ □□□□□ □□□□□□□□□
    0.01, # □: □□□□□□□□□□□ □□□□□□□□□ □□□□ (□□□□□) □ □□□□□□□□
    0.3]     # □: □□□□□□□□□□ □□□□□□□□
```

```
# □□□□□□□□□ □□□□□□□: [□□□□□□, □□□□□□□]
u0_lv = [40.0, 9.0] # □□□□□□□□□ □□□□□□□□□

# □□□□□□□□□ □□□□□□□□□
tspan_lv = (0.0, 200.0) # □□□□□□□□□□□□ □□□□□□□□□
```

```
dt_lv = 0.01 # □□□ □□□□□□□□□□□□□□

# □□□□□□□□ □ □□□□□□□ □□□□□□
prob_lv = ODEProblem(lotka_volterra!, u0_lv, tspan_lv, p_lv)
sol_lv = solve(prob_lv,
        dt = dt_lv,
        Tsit5(),# □□□□□ 5-□□ □□□□□□□
        reltol=1e-8,# □□□□□□□□□□□□□ □□□□□□□□
        abstol=1e-10,# □□□□□□□□□□ □□□□□□□□
        saveat=0.1,# □□□□□□□□□ □□□□□□ 0.1 □□□□□□□ □□□□□□□
        dense=true # □□□□□□□□ □□□□□□□ □□□□□ □□□ □□□□□□□□□□□□
)

# □□□□□□□□□□ □□□□□□
df_lv = DataFrame()
df_lv[!, :t] = sol_lv.t
df_lv[!, :prey] = [u[1] for u in sol_lv.u] # □□□□□□
df_lv[!, :predator] = [u[2] for u in sol_lv.u] # □□□□□□□

# □□□□□□□ □□□□□□□□□□□ □□□ □□□□□□□
df_lv[!, :dprey_dt] = p_lv[1] .* df_lv.prey .- p_lv[2] .* df_lv.prey
df_lv[!, :dpredator_dt] = p_lv[3] .* df_lv.prey .* df_lv.predator .

# □□□□□ □□□□□□□□□□ □ □□□□□□
println("="^60)
println("□□□□□□ □□□□□-□□□□□□□□□ (□□□□□□-□□□□□□)")
println("="^60)
println("\n□□□□□□□□□ □□□□□□:")
```

```
println("□ (□□□□□□□□ □□□□□□□□□□□ □□□□□) = ", p_lv[1])
println("□ (□□□□□□□□ □□□□□□□□ □□□□□) = ", p_lv[2])
println("□ (□□□□□□□□□□□ □□□□□□□□□) = ", p_lv[3])
println("□ (□□□□□□□□□□ □□□□□□□□) = ", p_lv[4])
println("\n□□□□□□□□□ □□□□□□□:")
println("□□□□□□ (x0) = ", u0_lv[1])
println("□□□□□□□ (y0) = ", u0_lv[2])

# □□□□□□□□□□□□ □□□□□ (□□□□□□□ □□□□□□□□)
x_star = p_lv[4] / p_lv[3] # □□□□□□□□□□□□ □□□□□ □□□ □□□□□
y_star = p_lv[1] / p_lv[2] # □□□□□□□□□□□□ □□□□□ □□□ □□□□□□□□
println("\n□□□□□□□□□□□□ □□□□□ (□□□□□□□□□ □□□□□□□□□□):")
println("x* = □/□ = ", round(x_star, digits=3))
println("y* = □/□ = ", round(y_star, digits=3))

# □□□□□□□□□□ □□□□□□□□
# □□□□□□ 1: □□□□□□□□ □□□□□□□□□ □□ □□□□□□□
plt1 = plot(df_lv.t, [df_lv.prey df_lv.predator],
    label=[L"□□□□□□ (x)" L"□□□□□□□ (y)"],
    xlabel="□□□□□",
    ylabel="□□□□□□□□□",
    title="□□□□□□ □□□□□-□□□□□□□□□: □□□□□□□□ □□□□□□□□□",
    linewidth=2,
    legend=:topright,
    grid=true,
    size=(900, 500),
    color=[:green :red])
```

```
# □□□□□□□□□□ □□□□□□□□□□□□ □□□□□□□
hline!(plt1, [x_star], color=:green, linestyle=:dash, alpha=0.5, la
hline!(plt1, [y_star], color=:red, linestyle=:dash, alpha=0.5, labe

# □□□□□□ 2: □□□□□□□ □□□□□□□ (□□□□□□□ vs □□□□□□)
plt2 = plot(df_lv.prey, df_lv.predator,
    label="□□□□□□□ □□□□□□□□□□",
    xlabel="□□□□□□□□□ □□□□□ (x)",
    ylabel="□□□□□□□□□ □□□□□□□□ (y)",
    title="□□□□□□□ □□□□□□□ □□□□□□□",
    color=:blue,
    linewidth=1.5,
    grid=true,
    size=(800, 600),
    legend=:topright)

# □□□□□□□□□□ □□□□□□□ □□□□□□□□□□□ □□ □□□□□□□ □□□□□□□□
step = 50 # □□□ □□□ □□□□□□□□□□□ □□□□□□□
for i in 1:step:length(df_lv.prey)-step
    plot!(plt2, [df_lv.prey[i], df_lv.prey[i+step]],
        [df_lv.predator[i], df_lv.predator[i+step]],
        arrow=:closed, color=:blue, alpha=0.3, label=false)
end

# □□□□□□□□□□ □□□□□□□□□□□□ □□□□□
scatter!(plt2, [x_star], [y_star],
    color=:black, markersize=8, label="□□□□□□□□□□□□ □□□□□ (x*, y*)")
```

```
# □□□□□□□□ (□□□□□□□□ □□□□□)
x_range = LinRange(0, maximum(df_lv.prey)*1.1, 100)
y_nullcline = p_lv[1] ./ (p_lv[2] .* x_range) # y-□□□□□□□□ (dy/dt =0)
plot!(plt2, x_range, y_nullcline,
    color=:red, linestyle=:dash, linewidth=1.5, label="□□□□□□□□ □□□□□□
    y_range = LinRange(0, maximum(df_lv.predator)*1.1, 100)
    x_nullcline = p_lv[4] ./ (p_lv[3] .* ones(length(y_range))) # x
plot!(plt2, x_nullcline, y_range,
    color=:green, linestyle=:dash, linewidth=1.5, label="□□□□□□□□ □□□□

# □□□□□□ 3: □□□□□□□□□□□ (□□□□□□□□ □□□□□□□□□)
plt3 = plot(df_lv.t, [df_lv.dprey_dt df_lv.dpredator_dt],
    label=[L"dx/dt" L"dy/dt"],
    xlabel="□□□□□",
    ylabel="□□□□□□□□ □□□□□□□□□",
    title="□□□□□□□□□□□ □□□□□□□□□",
    linewidth=1.5,
    legend=:topright,
    grid=true,
    size=(900, 400),
    color=[:green :red])
    hline!(plt3, [0], color=:black, linestyle=:solid, alpha=0.3,lab

# □□□□□□ 4: □□□□□□□□□□□□□ □□□□□□□□□ (□ %)
df_lv[!, :prey_pct_change] = df_lv.dprey_dt ./ df_lv.prey .* 100
df_lv[!, :predator_pct_change] = df_lv.dpredator_dt ./df_lv.predato

plt4 = plot(df_lv.t, [df_lv.prey_pct_change df_lv.predator_pct_chan
```

```
    label=[L"dx/dt / x (\%)" L"dy/dt / y (\%)"],
    xlabel="□□□□□",
    ylabel="□□□□□□□□□□□□□ □□□□□□□□□□, %",
    title="□□□□□□□□□□□□□ □□□□□ □□□□□",
    linewidth=1.5,
    legend=:topright,
    grid=true,
    size=(900, 400),
    color=[:green :red])

# □□□□□□ 5: □□□□□□□□□□□□□ □□□□□□ (□□□□□□□ □□□□□□□□□□□□□□□ □□□□□)
function compute_fft(signal, dt)
    n = length(signal) # □□□□□□□□□□□ rfft □□□ □□□□□□□□□□□□□ □□□□□□□□□ (□□□□□□□□□□
    
    spectrum = abs.(rfft(signal))
# □□□□□□□□□□□□□□□□ □□□□□□□ □□□ rfft
    freq = rfftfreq(n, 1/dt)
    return freq, spectrum
end

# □□□□□□□□□□ □□□□□□□□□ □□□□□□□□□□
freq_prey, spectrum_prey = compute_fft(df_lv.prey .-mean(df_lv.prey
freq_predator, spectrum_predator = compute_fft(df_lv.predator .-mea
plt5 = plot(freq_prey, [spectrum_prey spectrum_predator],
    label=[L"□□□□□□ (x)" L"□□□□□□□ (y)"],
    xlabel="□□□□□□□",
    ylabel="□□□□□□□□□□",
    title="□□□□□□□□□□□□□ □□□□□□ (□□□□□)",
```

```
    linewidth=1.5,
    xscale=:log10,
    yscale=:log10,
    legend=:topright,
    grid=true,
    size=(800, 400),
    color=[:green :red])

# □□□□□□□□□□ □□□□□□□□□□□□ □□□□□□
if length(spectrum_prey) > 0
    idx_prey = argmax(spectrum_prey[2:end]) + 1 # □□□□□□□□□□ □□□□□□□ □□□
    dominant_freq_prey = freq_prey[idx_prey]
    period_prey = 1/dominant_freq_prey
    println("\n□□□□□□□□□□□□ □□□□□□□ □□□□□□□□□ □□□□□: ", round(dominant_freq
    println("□□□□□□ □□□□□□□□□ □□□□□: ", round(period_prey, digits=2), "
end

# □□□□□□ 6: □□□□□□□□□□ □□□□□□ □□□□ □□□□□□□□
plt6 = plot(layout=(3, 2), size=(1200, 900))
plot!(plt6[1], df_lv.t, df_lv.prey, label=L"x(t)", color=:green, li
plot!(plt6[2], df_lv.t, df_lv.predator, label=L"y(t)", color=:red,
plot!(plt6[3], df_lv.prey, df_lv.predator, label=false, color=:blue
scatter!(plt6[3], [x_star], [y_star], color=:black, markersize=5, la
plot!(plt6[4], df_lv.t, [df_lv.dprey_dt df_lv.dpredator_dt], label=
plot!(plt6[5], freq_prey, spectrum_prey, label=L"x", color=:green,
plot!(plt6[6], df_lv.t, [df_lv.prey_pct_change df_lv.predator_pct_c

# □□□□□□ □□□□□□□□□□□
```

```
println("\n" * "="^60)
println("□□□□□□ □□□□□□□□□□□")
println("="^60)
println("\n□□□□□□□□ □□□□□□□□□□:")
println("□□□□□□: min = ", round(minimum(df_lv.prey), digits=2),
    ", max = ", round(maximum(df_lv.prey), digits=2),
    ", mean = ", round(mean(df_lv.prey), digits=2))
println("□□□□□□□: min = ", round(minimum(df_lv.predator), digits=2),
    ", max = ", round(maximum(df_lv.predator), digits=2),
    ", mean = ", round(mean(df_lv.predator), digits=2))

# □□□□□□□□□□ □□□□□□ □□□□□□□□□ □□□ □□□□□□ □□□□□□□□□□
# □□□□□□ □□□□□□□□ □□□□□□□ □□□□□□ □□□, □□□□□□ □□□□□□□□□ □□□□□□□□ □□□□□□□□□□□□□□□
# □□□□□□□ □□□□□ □□□□□□□ □□□□ □□□□□ (□□□□□□□ □□□□□□□□)

function find_first_peak(signal, time)
    for i in 2:length(signal)-1
        if signal[i] > signal[i-1] && signal[i] > signal[i+1]
            return time[i], signal[i]
        end
    end
    return NaN, NaN
end

peak_time_prey, peak_value_prey = find_first_peak(df_lv.prey, df_lv
peak_time_predator, peak_value_predator = find_first_peak(df_lv.pre

if !isnan(peak_time_prey) && !isnan(peak_time_predator)
```

```
    phase_shift = peak_time_predator - peak_time_prey
    println("\n□□□□□□ □□□□□□□□□:")
    println("□□□□□□ □□□ □□□□□: □□□□□ = ", round(peak_time_prey,digits=2)
    println("□□□□□□ □□□ □□□□□□□□: □□□□□ = ", round(peak_time_predator, d
    println("□□□□□ □□□ (□□□□□□□ □□□□□□□): ", round(phase_shift, digits=2
end

# □□□□□□□□□□ □□□□□□□□
savefig(plt1, plotsdir(script_name, "lv_dynamics.png"))
savefig(plt2, plotsdir(script_name, "lv_phase_portrait.png"))
savefig(plt3, plotsdir(script_name, "lv_derivatives.png"))
savefig(plt4, plotsdir(script_name, "lv_relative_changes.png"))
savefig(plt5, plotsdir(script_name, "lv_spectrum.png"))
savefig(plt6, plotsdir(script_name, "lv_panel.png"))

# □□□□□□□□□□□□□□ □□□□□□: □□□□□□□□□□□□□□□ □ □□□□□□□□□□
println("\n\n" * "="^60)
println("□□□□□□ □□□□□□□□□□□□□□□")
println("="^60)

# □□□□□□□ □□□ □□□□□□□ □□□□□□□ □□□□□□□□□□
function analyze_parameter_sensitivity(param_index, values, param_na
    println("\n□□□□□□ □□□□□□□□□□□□□□□ □ □□□□□□□□□: ", param_name)
    results = []
    for val in values
    p_test = copy(p_lv)
    p_test[param_index] = val
    prob_test = ODEProblem(lotka_volterra!, u0_lv, tspan_lv, p_test
```

```
    sol_test = solve(prob_test, dt = dt_lv)
    prey_end = sol_test.u[end][1]
    predator_end = sol_test.u[end][2]
    push!(results, (param=val, prey=prey_end, predator=predator_end
    println(" $(param_name)=$(val): □□□□□□=$(round(prey_end,2)), □□□□
    end
    return results
end

# □□□□□□ □□□□□□□□□□□□□□□□ □ □□□□□□□□ □□□□□□□□□□
if false # □□□□□□□□□□ □ true □□□ □□□□□□□□□□ □□□□□□□
    println("\n1. □□□□□□□ □□□□□□□□ □□□□□□□□□□□ □□□□□ (□):")
    analyze_parameter_sensitivity(1, [0.05, 0.1, 0.2, 0.3], "□")
    println("\n2. □□□□□□□ □□□□□□□□□□ □□□□□□□□ (□):")
    analyze_parameter_sensitivity(4, [0.1, 0.3, 0.5, 0.7], "□")
end

println("\n□□□□□□□□□□□□□ □□□□□□□□□ □□□□□□□!")
```

Выполнила код и сгенерировала чистый код, документацию в формате Quarto, jupyter notebook. Далее я проверила работу в jupyter и сохранила графики:

Производные популяции показывают изменение популяции жертв и хищников

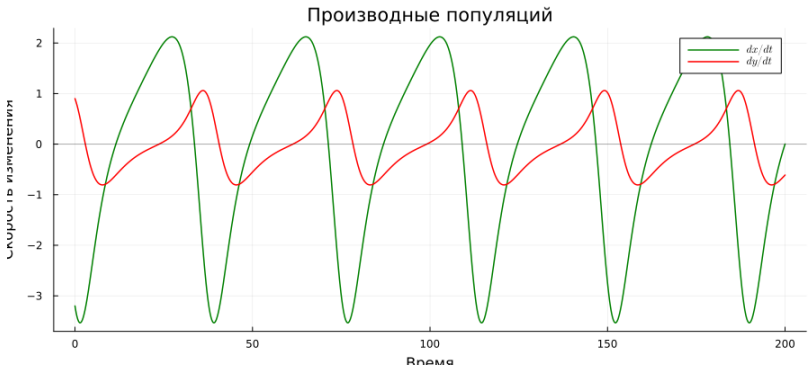

Рисунок 4.12: Производные популяции

Динамика популяции по времени

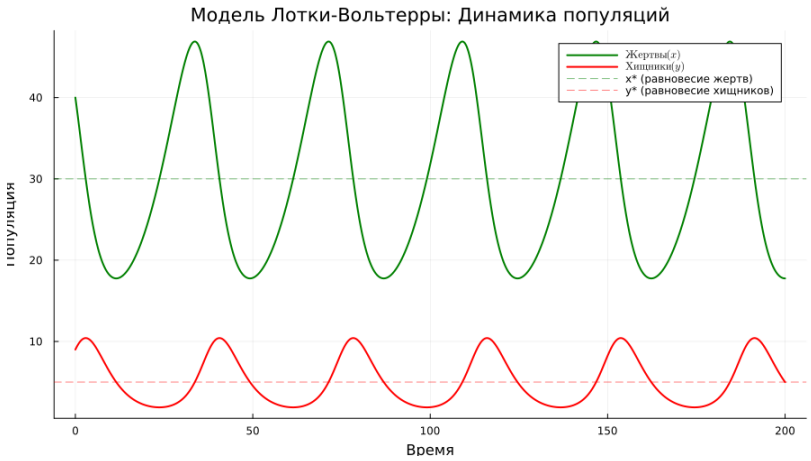

Рисунок 4.13: Динамика популяции

Фазовый портрет системы

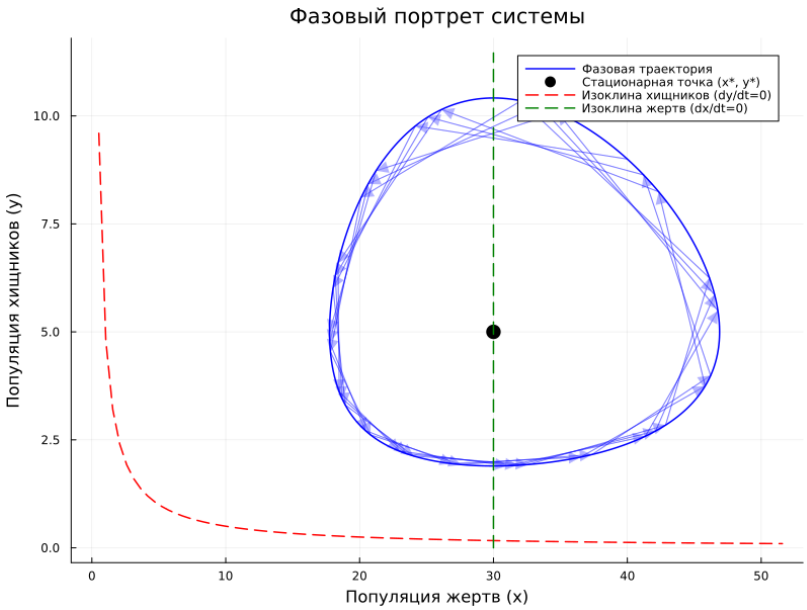

Рисунок 4.14: Фазовый портрет lv

Темпы роста популяции относительно друг другу

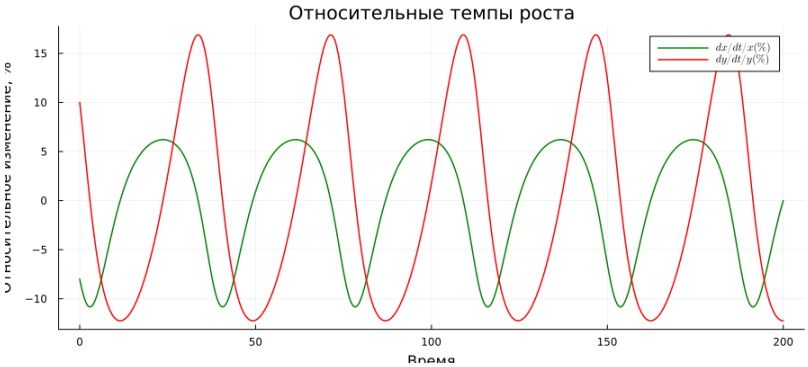

Рисунок 4.15: Относительные темпы роста

Спектральный анализ роста

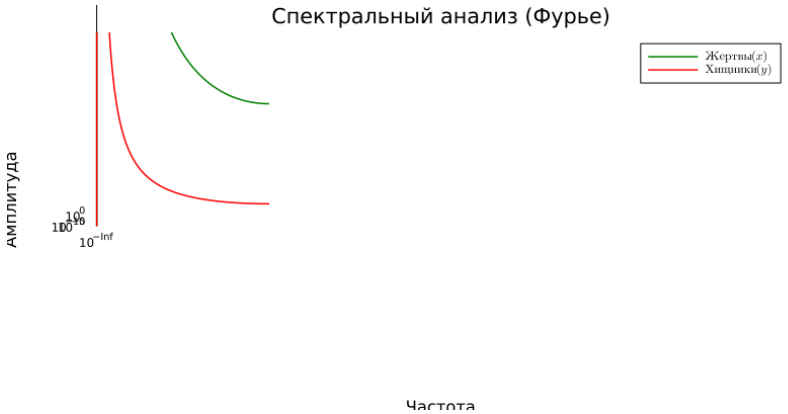

Рисунок 4.16: Спектральный анализ

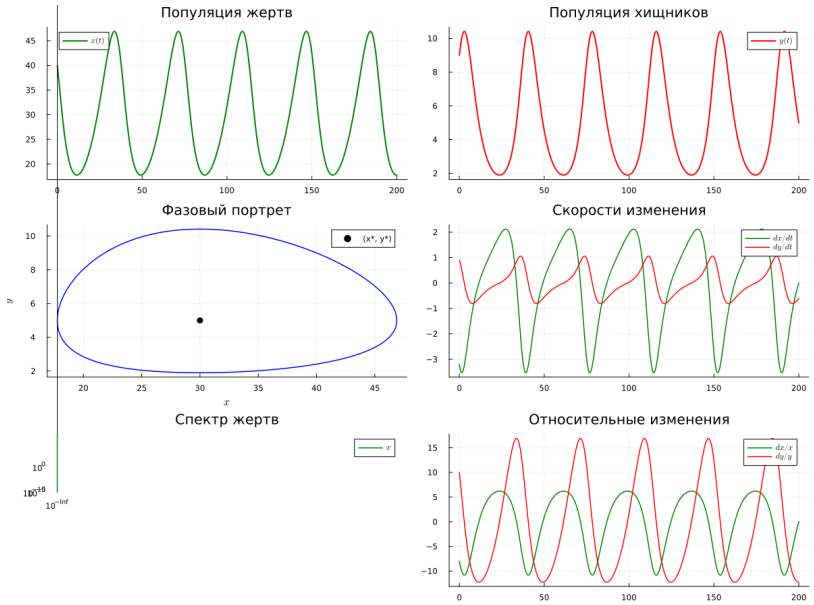

Рисунок 4.17: Итоговая график lv

В литературном коде, я добавила набор параметров

```
# [illegible]: [α, β, γ, δ]
parameter_sets = [
    [0.20, 0.06, 0.04, 0.10],  # [illegible] (α=0.20)
    [0.30, 0.06, 0.04, 0.10],  # [illegible] α=0.30
```

```
    [0.10, 0.06, 0.04, 0.10],  # □□□□□□□□□□□ □□□□ □□□□□
    [0.20, 0.08, 0.04, 0.10],  # □□□□□□□□□□□ □□□□□□□□ □□□□□
    [0.20, 0.04, 0.04, 0.10],  # □□□□□□□□□□□ □□□□□□□□ □□□□□
    [0.20, 0.06, 0.06, 0.10],  # □□□□□□□□□□□ □□□□□□□□□ □ □□□□□□□□
    [0.20, 0.06, 0.02, 0.10],  # □□□□□□□□□□□ □□□□□□□□□ □ □□□□□□□□
    [0.20, 0.06, 0.04, 0.15],  # □□□□□□□□□□□ □□□□□□□□□□ □□□□□□□□
    [0.20, 0.06, 0.04, 0.05],  # □□□□□□□□□□□ □□□□□□□□□□ □□□□□□□□
]

# □□□□□□□□□ □□□ □□□□□□□□□□
results = []

# □□□□□□□ □□□ □□□□□□□□□□ □□□□□□□ □□□□
function find_first_peak(signal, time)
    for i in 2:length(signal)-1
        if signal[i] > signal[i-1] && signal[i] > signal[i+1]
            return time[i], signal[i]
        end
    end
    return NaN, NaN
end

# □□□□□□□ □□□ □□□□□□□□□□ □□□□□□□
function compute_fft(signal, dt)
    n = length(signal)
    spectrum = abs.(rfft(signal))
    freq = rfftfreq(n, 1/dt)
    return freq, spectrum
```

```
end

# □□□□□□□□ □□□□□□ □□ □□□□□□□ □□□□□□ □□□□□□□□□□
for (idx, p) in enumerate(parameter_sets)
    println("\n" * "="^60)
    println("□□□□□ □□□□□□□□□□ $idx")
    println("="^60)

    # □□□□□ □□□□□□□□□□
    println("□□□□□□□□□ □□□□□□:")
    println("□ (□□□□□□□□ □□□□□□□□□□ □□□□□) = ", p[1])
    println("□ (□□□□□□□□ □□□□□□□□ □□□□□) = ", p[2])
    println("□ (□□□□□□□□□□ □□□□□□□□□) = ", p[3])
    println("□ (□□□□□□□□□□ □□□□□□□□□) = ", p[4])

    # □□□□□□□□□□□□ □□□□□ (□□□□□□□ □□□□□□□□)
    x_star = p[4] / p[3] # □□□□□□□□□□□□ □□□□□ □□□ □□□□□
    y_star = p[1] / p[2] # □□□□□□□□□□□□ □□□□□ □□□ □□□□□□□□
    println("\n□□□□□□□□□□□□ □□□□□ (□□□□□□□□□ □□□□□□□□□□):")
    println("x* = □/□ = ", round(x_star, digits=3))
    println("y* = □/□ = ", round(y_star, digits=3))

    # □□□□□□□□ □ □□□□□□□ □□□□□□
    prob_lv = ODEProblem(lotka_volterra!, u0_lv, tspan_lv, p)
    sol_lv = solve(prob_lv,
                   dt = dt_lv,
                   Tsit5(),# □□□□□ 5-□□ □□□□□□□
                   reltol=1e-8,# □□□□□□□□□□□□□ □□□□□□□□
```

```
                    abstol=1e-10,# □□□□□□□□□□ □□□□□□□□
                    saveat=0.1,# □□□□□□□□□ □□□□□□ 0.1 □□□□□□□ □□□□□□□
                    dense=true) # □□□□□□□□ □□□□□□□ □□□□□ □□□ □□□□□□□□□□□□

    # □□□□□□□□□□ □□□□□□
    df_lv = DataFrame()
    df_lv[!, :t] = sol_lv.t
    df_lv[!, :prey] = [u[1] for u in sol_lv.u] # □□□□□□
    df_lv[!, :predator] = [u[2] for u in sol_lv.u] # □□□□□□□
    df_lv[!, :□□□□□] .= "□□□□□ $idx"

    # □□□□□□ □□□□□□□□□□□ □□□ □□□□□□□
    df_lv[!, :dprey_dt] = p[1] .* df_lv.prey .- p[2] .* df_lv.prey
    df_lv[!, :dpredator_dt] = p[3] .* df_lv.prey .* df_lv.predator

    # □□□□□□□□□□□□□ □□□□□□□□□
    df_lv[!, :prey_pct_change] = df_lv.dprey_dt ./ df_lv.prey .* 10
    df_lv[!, :predator_pct_change] = df_lv.dpredator_dt ./ df_lv.pre

    # □□□□□□□□□ □□□□□□□□□□□ □□□□□ □ DataFrame
    df_lv[!, :x_star] .= x_star
    df_lv[!, :y_star] .= y_star

    # □□□□□□□□□ □□□□□□□□□□
    push!(results, df_lv)

    # □□□□□□□□ □□□□□□□□□□
    println("\n□□□□□□□□ □□□□□□□□□□:")
```

```
    println("□□□□□□: min = ", round(minimum(df_lv.prey), digits=2),
            ", max = ", round(maximum(df_lv.prey), digits=2),
            ", mean = ", round(mean(df_lv.prey), digits=2))
    println("□□□□□□□: min = ", round(minimum(df_lv.predator), digits=
            ", max = ", round(maximum(df_lv.predator), digits=2),
            ", mean = ", round(mean(df_lv.predator), digits=2))

    # □□□□□□ □□□□□□□□□
    peak_time_prey, peak_value_prey = find_first_peak(df_lv.prey, d
    peak_time_predator, peak_value_predator = find_first_peak(df_lv

    if !isnan(peak_time_prey) && !isnan(peak_time_predator)
        phase_shift = peak_time_predator - peak_time_prey
        println("\n□□□□□□ □□□□□□□□:")
        println("□□□□□□ □□□ □□□□□: □□□□□ = ", round(peak_time_prey, digi
                ", □□□□□□□□ = ", round(peak_value_prey, digits=2))
        println("□□□□□□ □□□ □□□□□□□: □□□□□ = ", round(peak_time_predato
                ", □□□□□□□□ = ", round(peak_value_predator, digits=2)
        println("□□□□□ □□□ (□□□□□□□ □□□□□□□): ", round(phase_shift, digi
    end

    # □□□□□□□□□□□□ □□□□□□
    freq_prey, spectrum_prey = compute_fft(df_lv.prey .- mean(df_lv
    if length(spectrum_prey) > 0
        idx_prey = argmax(spectrum_prey[2:end]) + 1
        dominant_freq_prey = freq_prey[idx_prey]
        period_prey = 1/dominant_freq_prey
        println("□□□□□□□□□□□ □□□□□□ □□□□□□□□□ □□□□□: ", round(period_prey
```

```
    end
end
```

Проверила статистику и работу в jupyter

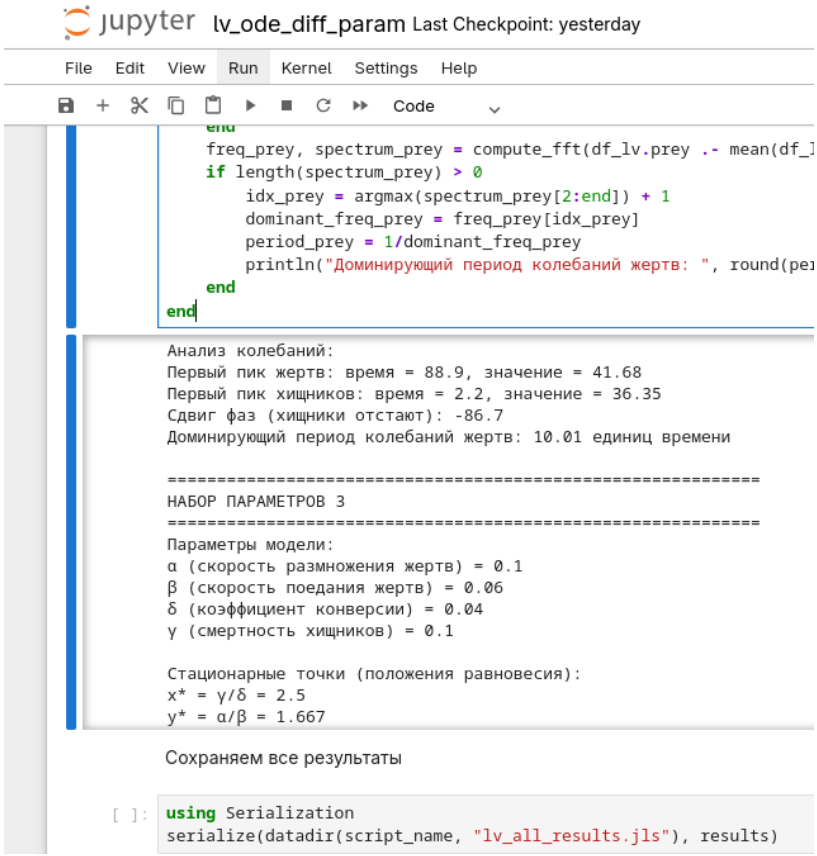

Рисунок 4.18: Статистика модели lv с набором параметров

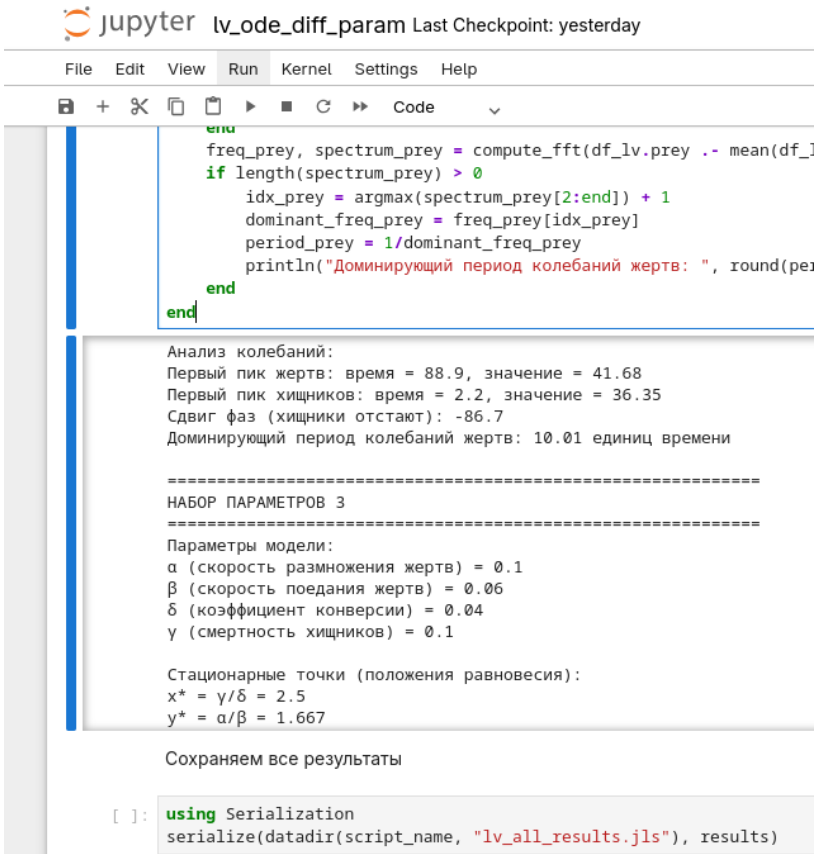

Рисунок 4.19: Работа в jupyter

Я сравнивала фазовых портретов для всех параметров

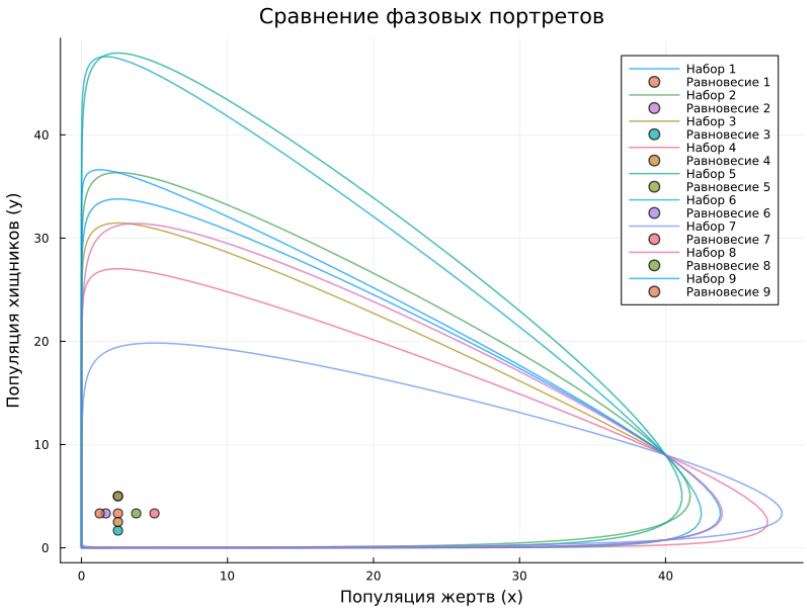

Рисунок 4.20: Сравнение фазовых портретов

Сравнивала популяции жертв при разных параметров

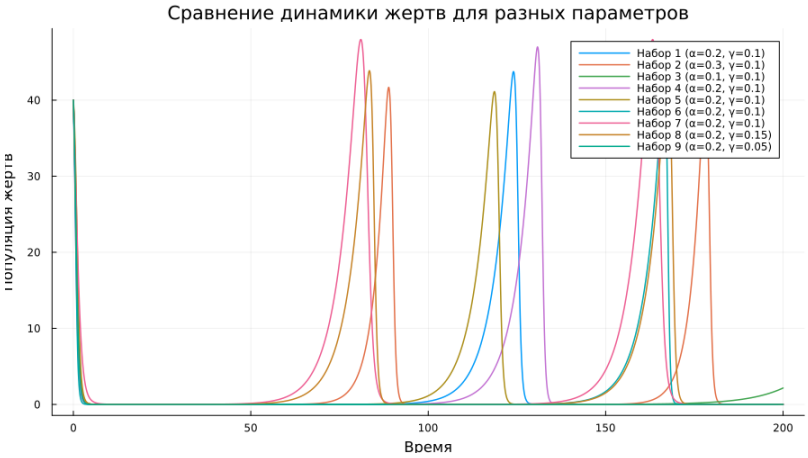

Рисунок 4.21: Сравнение попляции жертв

Сравнивала популяции жищков при разных параметров

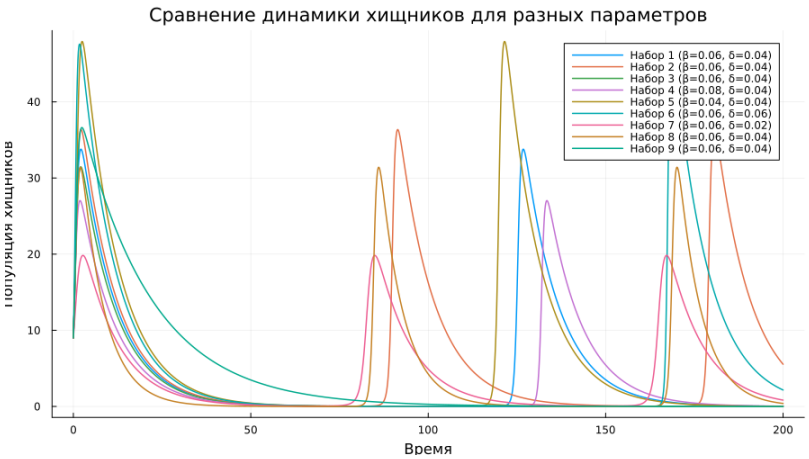

Рисунок 4.22: Сравнение популяции жищков

5 Выводы

При выполнении данной работы я научилась работать с моделями SIR и Лотки-Вольтерры и освоила использование профессиональными инструментами.

Список литературы